

EncryptoVision: A dual-modal fusion-based multi-classification model for encrypted traffic recognition

Zhiyuan Li ^{a,b,c,*}, Yujie Jin ^a

^a The College of Computer Science and Telecommunication Engineering, Jiangsu University, Zhenjiang, 212013, China

^b Jiangsu Provincial Key Laboratory of Industrial Network Security Technology, Zhenjiang, Jiangsu, 212013, China

^c Jiangsu Ubiquitous Data Intelligent Perception and Analysis Application Engineering Research Center, Zhenjiang, Jiangsu, 212013, China

ARTICLE INFO

Keywords:

Encrypted traffic recognition
Feature extraction
Spatial-temporal fusion
Vision transformer
Channel attention

ABSTRACT

With the development of security, confidentiality, and data privacy technologies, the classification of fine-grained encrypted traffic has become increasingly important. Nowadays, existing deep learning methods, including CNN, LSTM, and transformer, have shown impressive classification performance. However, many of these methods merely utilize the raw packet bytes to generate traffic representations, resulting in the potential loss of crucial information, such as dynamic traffic patterns and changes in protocols. In this paper, we propose a dual-modal fusion-based multi-classification model for encrypted traffic recognition, called EncryptoVision. Firstly, we transform the encrypted traffic data into three-channel images and incorporate a triplet attention mechanism to enhance the interaction among the three channels. Then, we use the multi-head self-attention mechanism to expand the model's global receptive field, allowing it to capture more detailed spatial feature information. Additionally, we also leverage the learning abilities of the transformer encoder to extract temporal feature information from the traffic for long-term time series prediction. Next, we use the spatial-temporal fusion features to obtain the fine-grained features for multi-classification. Experimental results show that our model outperforms state-of-the-art models in classification performance across four real-world encrypted traffic datasets.

1. Introduction

In the era of big data, people are increasingly concerned about their security and privacy in cyberspace. Encryption technology has been widely applied in communication and various applications, such as QUIC, VPN and Tor [1]. These applications and services can strengthen the anonymity and security. But sometimes, malicious users use the encryption and privacy protection technologies to launch the DDoS attack or exploit the dark web to conduct illegal transactions [2]. In view of this, it is necessary to study the regulation and analysis of encrypted network traffic. Nowadays, encrypted traffic identification exists some challenges, such as visibility issues, dynamic patterns, protocol diversity, privacy and compliance and feature extraction. Especially, the combination of dynamic patterns and protocol diversity reduced the effectiveness of feature extraction and multi-applications classifications.

Recently, there exist some machine learning methods, such as support vector machines (SVM), decision trees, deep learning and transformer technologies to address encrypted traffic classification problems. In the presented methods, some models based on neural networks

(convolutional neural network (CNN), long short-term memory network (LSTM) and transformer) significantly improve the accuracy of encrypted traffic classification [3]. These methods can automatically learn traffic representations and exhibit strong robustness. However, most of these methods focus primarily on optimizing the model itself, lacking improvements in the modeling approach for encrypted traffic. The most critical encrypted component in encrypted traffic is the application layer data, which refers to the actual content transmitted by users while using network services (such as file uploads or downloads). Existing traffic representation methods can result in the information from other packets being overwhelmed by a single excessively long packet [4]. Besides, some malicious encrypted flows may include obfuscation information in the first packet to conceal subsequent attack behaviors. What is more, existing methods are inadequate in extracting spatial features, resulting in a limited in-depth analysis of encrypted traffic. For example, the receptive field of CNN is constrained by the fixed size of the convolutional kernels, while the transformer-based methods divide traffic flow images into fixed-size patches and treat them as sequence elements. These feature extraction

* Corresponding author.

E-mail addresses: lizhiyuan@ujs.edu.cn (Z. Li), 2222308044@stmail.ujs.edu.cn (Y. Jin).

<https://doi.org/10.1016/j.comnet.2025.111499>

Received 4 March 2025; Received in revised form 20 June 2025; Accepted 22 June 2025

Available online 7 July 2025

1389-1286/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

methods can lead to the loss of local feature information. Therefore, optimizing traffic representation methods and proposing a model for classifying encrypted traffic is a top priority in current encrypted traffic classification efforts.

L2-BiTCN-CNN [5] is a dual-branch model that considers both spatial and temporal features, which provides a solid foundation for our model structure. YaTC [6] introduces a Multi-Feature Representation (MFR) method for traffic representation, which inspired our three-channel traffic image representation. ATViTSC [7] uses a transformer encoder to extract features from traffic images, and we use this method in our spatial feature extraction module as well. FS-Net [8] highlights the significance of extracting temporal features from encrypted traffic sequences. ET-BERT [9] converts encrypted traffic data into semantic sequence representations and adds embedding vectors for classification. Both models inspired our temporal feature extraction module. Based on these studies, this paper proposes EncryptoVision model for encrypted traffic classification. We firstly transforms encrypted traffic into three-channel images representation and studies the use of channel attention mechanism to enhance the interaction between encrypted traffic data, combined with multi-head self attention mechanism (MSA) to capture the sufficient and necessary features. By incorporating channel attention mechanism, we substantially enhance model's ability to capture key features, diminishing the influence of irrelevant and redundant features on classification performance. The main contributions of this paper are summarized as follows:

- The proposed model, called EncryptoVision, is designed to identify different types of encrypted network traffic. It takes three-dimensional traffic images and byte sequences generated from encrypted traffic as input, extracts spatial and temporal features from these two modalities, and then generates spatial-temporal features for classification.
- This paper proposes a traffic representation method that transforms encrypted traffic into a three-channel images. This approach effectively leverages packet-level or flow-level features within network flows. Additionally, we employed a triplet attention for data augmentation, allowing the model to learn more meaningful internal representations of the traffic image.
- The experiment evaluated the performance of the model and the improvement of the model's classification performance with the triplet attention. The experimental results on four publicly available encrypted traffic datasets demonstrate that transforming encrypted traffic into three-channel images and incorporating the triplet attention mechanism notably enhances the model's classification performance.

The highlight of our work lies in the feature extraction method, as shown in Fig. 1. In the figure, the network flow consists of header information and the payload, which is represented by the leftmost rounded rectangle. We can extract the flow-level or packet-level matrix from the network traffic. The previous feature extraction technologies includes the natural language processing (NLP) and CNN methods. The NLP methods can transform packet segments into 16-bit token units, such as $0 \times 234d$ or $0 \times 1dd0$. The CNN methods can convert the traffic data into feature maps. Our method can consider both spatial and temporal features, and combine them to obtain a more comprehensive feature representation.

The remainder of this work is organized as follows: Section 2 summarizes recent works about encrypted traffic recognition and classification. Section 3 presents the EncryptoVision model architecture and also gives its detailed descriptions. Section 4 provides a comprehensive performance evaluation, including parameter sensitivity analysis, comparative results with the state-of-the-art models. Section 4.1 gives some discussion about our proposed method. Section 5 concludes the paper and provides details about future work.

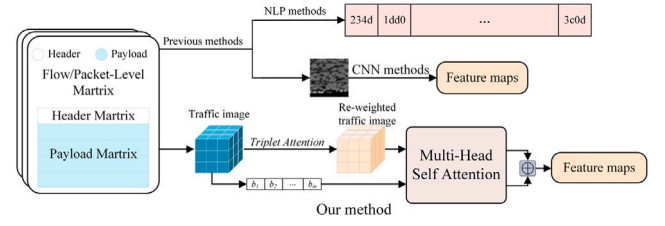


Fig. 1. The comparison of feature extraction between our model and previous methods.

2. Related works

This section provides an overview of research on encrypted network traffic classification, including machine learning-based, deep learning-based, and transformer-based methods.

2.1. Machine learning-based methods

Some encrypted traffic classification methods such as port-based [10], protocol-based [11] and content-based [12] methods have limited flexibility and accuracy, and therefore are not successful in recognizing various encrypted traffic. After that, machine learning methods [13], including supervised learning [14] and reinforcement learning [15] are applied to identify various encrypted traffic. During this period, some machine learning-based models [13] achieved better classification and recognition results, including but not limited to naive bayes [16], support vector machines (SVM) [17] and decision trees [18]. Each model has its own merits, such as the simplicity and speed of naive bayes, the interpretability of decision trees, and the strong generalization ability of SVM. In most cases, machine learning utilizes supervised learning techniques that require pre-labeled training datasets. These datasets are used to train the models to differentiate various traffic categories. Although machine learning-based traffic classification has made progress, it still faces challenges related to the complexity of feature engineering and the need for manual intervention. It also requires greater flexibility and resilience to handle highly encrypted traffic and continuously evolving network applications.

2.2. Deep learning-based methods

The emergence of deep learning technologies [19] has significantly reduced the need for manual feature engineering in traffic classification. Models such as CNN [20,21], recurrent neural network (RNN) [22], LSTM [23] can immediately learn effective feature representations from encrypted traffic without the requirement for manual feature construction. Deep learning models can effectively capture intricate patterns and relationships in encrypted network traffic, enabling them to outperform other approaches in traffic classification tasks. Some researchers have combined attention mechanisms with classic deep learning models, leading to the development of methods such as CNN with attention [24], LSTM with attention [25], and others. The incorporation of attention mechanism has significantly improved the capacity of models to focus on crucial features and extract important features within the encrypted traffic data. The advancement of attention mechanisms has led to the development of neural networks that utilize channel attention mechanisms, such as SENet [26] and CBAM [27], which have achieved remarkable results in various object identification and image classification tasks. Channel attention mechanisms aim to explicitly represent the interdependencies across feature channels, automatically capturing the relevance of each feature through a learning process. Misra et al. [28] improved the triplet attention mechanism by enhancing data interaction among channels through pooling and convolution techniques.

Table 1
The data representation for the three channels of traffic images.

Channel	Dataset	
	USTC-TFC2016 ISCX-VPN2016 CIC-IOT2022	CESNET-QUIC22
$Channel_1$	Header(80), Payload(176)	SRC_IP(4),DST_IP(4),DST_ASN(1),SRC_PORT(2),DST_PORT(2), PROTOCOL(2),PHIST_SRC_SIZES(32),PHIST_DST_SIZES(32)
$Channel_2$	Header(80), Payload(176)	PHIST_SRC_IPT(32), PHIST_DST_IPT(32),PPI(variable length)
$Channel_3$	Header(80), Payload(176)	QUIC_VERSION(4),QUIC_SNI(1),DURATION(1),BYTES(2),BYTES_REV(2), PACKETS(2),PACKETS_REV(2),PPI_LEN(2),FLOW_ENDREASON_IDLE(1), PPI_DURATION(2),PPI_ROUNDTRIPS(2),FLOW_ENDREASON_ACTIVE(1) FLOW_ENDREASON_OTHER(1)

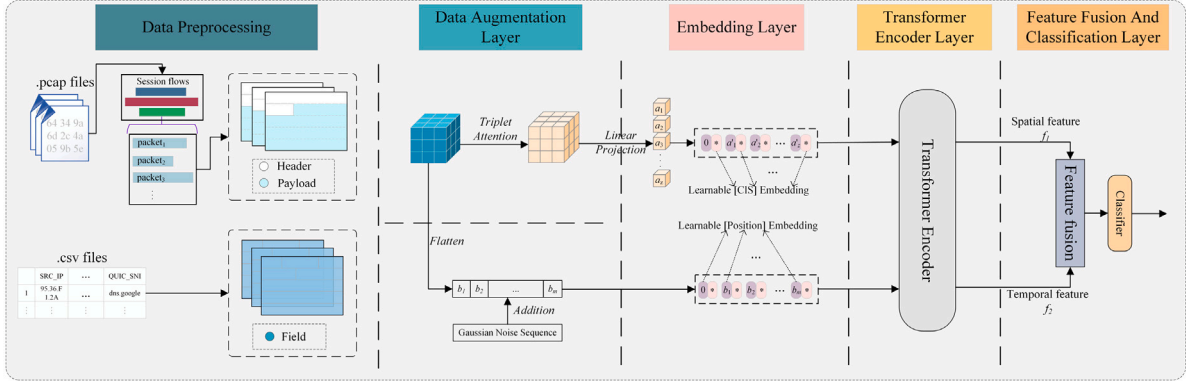


Fig. 2. The EncryptoVision model structure..

2.3. Transformer-based methods

Due to the transformer model's ability to effectively capture long-range dependencies and process sequential data [29], researchers have proposed many transformer-based methods recently. Cheng et al. [30] proposed a lightweight neural network specifically designed for online encrypted traffic classification. In [31], a novel transformer-based representation approach was proposed. In the data preprocessing stage, the encrypted traffic data was converted into single-channel images. The payloads were transformed into a format suitable for efficient learning by the transformer model through appropriate encoding techniques. Lin et al. [9] developed a technique that utilized a pre-trained transformer model to capture long-range dependencies and contextual interactions in byte sequences. This technique aims to analyze the behavioral patterns of encrypted network traffic in order to improve the accuracy of traffic classification. Zhao et al. [6] proposed a training method that randomly masked specific sections of the input packets during the pre-training phase. The training method can help the model understand the contextual relationships and traffic patterns within the byte sequences. This strategy significantly enhances the model's ability to express the inherent relationships and patterns within the traffic data. Liu et al. [7] proposed an encrypted traffic classification model based on vision transformer (ViT), which includes PVT and STFE modules for extracting global features and spatiotemporal features. The model also introduces a dynamic weighting mechanism to avoid the issue of unbalanced weight distribution during feature fusion.

Regarding the aforementioned challenges, we proposed a dual-modal fusion-based multi-classification model for encrypted network traffic and applications. By incorporating a triplet attention, we enhance the cross-dimensional interactions between the traffic data and mitigated the loss of spatial information caused by traffic reconstruction and patch segmentation. Furthermore, we used a transformer encoder to directly extract the temporal features from the encrypted traffic byte sequences. Finally, we combined the extracted spatial features and temporal features to obtain the spatial-temporal fusion features for multi-classification.

3. Our model

In this section, we introduce the structure of our proposed model, as well as the detailed processing steps for encrypted traffic, including encrypted network traffic preprocessing, spatial feature extraction, temporal feature extraction, feature fusion, and classification.

3.1. Encrypted network traffic preprocessing

The varying lengths and uninformative fields of the traffic data make it essential to preprocess the raw encrypted traffic before model training. The former three datasets used in this paper—USTC-TFC2016 [32], ISCX-VPN2016 [33], and CIC-IOT2022 [34]—are provided in .pcap file, containing all information about network flows and packets. In contrast, CESNET-QUIC22 [35] is provided in .csv file and contains only flow information and partial packet data. To effectively leverage the information in the traffic data and standardize the generation size of traffic images, we begin by performing traffic cleaning and segmentation on the .pcap files. This process is illustrated in the data preprocessing module of Fig. 2.

3.1.1. Traffic segmentation and three-dimensional matrix construction

For the USTC-TFC2016, ISCX-VPN2016 and CIC-IOT2022 datasets, it is essential to clean the raw traffic data before segmenting the raw traffic. We have used Wireshark to filter some noise traffic about the control and auxiliary protocols, such as ARP, STUN, DNS, ICMP, and so on. This is because the packets do not contain application payloads, and they are only used to support and maintain the network operation. Next, we can use the SplitCap to segment the raw traffic into the different network flows based on “network sessions”. Every network flow contains some data packets with the same TCP or UDP five-tuple packet header. Additionally, for the USTC-TFC2016, ISCX-VPN2016 and CIC-IOT2022 datasets, we have to deal with the segmented flow files with their filename extension .pcap. If a flow has more than three packets, we can directly extract the first three packets $[p_{i_1}, p_{i_2}, p_{i_3}]$. Otherwise, we have to add the fixed-format zero data firstly, and then extract the

first three packets. As shown in Table 1, the byte length of each packet will be either truncated or zero-padded to match a size of 16×16 , which includes a 20-byte IP header, a 20-byte TCP or UDP header (with 12 bytes of zero-padding for the UDP header for alignment), 40-byte of optional headers and a 176-byte payload. Subsequently, the matrix generated from the three packets are concatenated into a $3 \times 16 \times 16$ three-dimensional matrix representation M . But, the CESNET-QUIC22 dataset is different with the previous three datasets. There are some files with their filename extension .csv in the CESNET-QUIC22 dataset. As shown in Table 1, We use the 24 data fields for classification and allocate the corresponding data fields to the three channels of an image. Since the data values of the data fields are decimal, we can directly normalize the values of each data field to the range of $[0, 255]$. But, the “QUIC_SNI” field is stored as a string, so we apply MD5 hashing function on it and convert the hashing value into an integer value between 0 and 255. Next, we use the operations of zero-padding or trimming on the data values from each channel to get a fixed length of 256 bytes. After that, we can also convert the byte data from the flow into a three-dimensional matrix representation M . For the different flows, we apply the same preprocessing methods mentioned above to obtain the corresponding three-dimensional matrix representations.

3.1.2. Traffic transformation

Firstly, we transform the hexadecimal values of three-dimensional traffic matrix M into decimal values. This transformation method is to read two continuous hexadecimal values from the matrix and convert them into an integer ranging from 0 to 255. The integer from 0 to 255 is used as the pixel value for the traffic images. For the three-dimensional matrix generated from the CESNET-QUIC22 dataset, the traffic data have been converted to decimal with the same method in the Section 3.1.1. After that, the three-dimensional matrix M can be transformed to generate the corresponding traffic image $\mathcal{T}$ in .png format. Besides, this matrix will be flattened to produce a byte sequence $B = [b_1, b_2, \dots, b_m]$, where $m = 3 \times 16 \times 16$. Next, we normalize the pixel values from $[0, 255]$ to $[0, 1]$, as shown in Eq. (1). Both the matrices $\min(M)$ and $\max(M)$ are three-dimensional matrices with the same shape of M .

$$\mathcal{T} = \frac{M - \min(M)}{\max(M) - \min(M)} \quad (1)$$

where the matrix $\min(M)$ denotes the minimal matrix of the matrix M , and the matrix $\max(M)$ denotes the maximal matrix of the matrix M .

3.2. EncryptoVision model

The structure of our proposed dual-modal fusion-based multi-classification model is shown in Fig. 2. The training process of EncryptoVision model can be divided into a spatial feature extraction and a temporal feature extraction. For the spatial feature extraction, the EncryptoVision model processes the three-channel images $\mathcal{T}$. And then, the model can extract spatial relationships of the byte data from the three-channel images using the triplet attention and multi-head self-attention. For the temporal feature extraction, the model analyzes the byte sequences B to capture the temporal dependencies between them. The spatial-temporal features are obtained by concatenating the features generated for multi-classification.

3.2.1. Spatial feature extraction

In this section, the detailed descriptions about spatial feature extraction process are provided, including the triplet attention layer, the embedding layer and the transformer encoder layer.

1. Triplet attention layer

The input three-channel image $\mathcal{T} \in \mathbb{R}^{3 \times H \times W}$ is first sent to the triplet attention layer, which aims to enhance the interaction between the image channels through rotation operations and residual transformations. This operation promotes cross-dimensional interaction, enabling the neural network to adaptively learn and further prioritize the

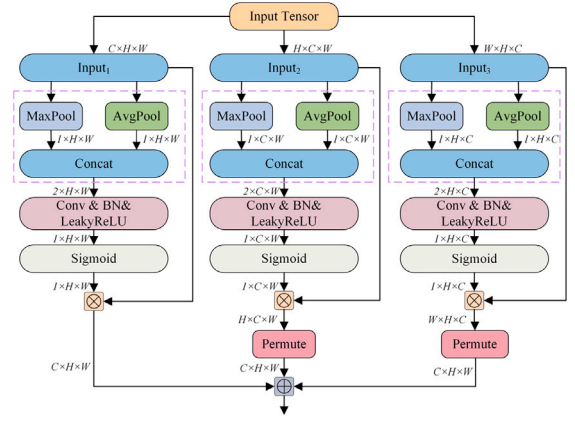


Fig. 3. The workflow of triplet attention layer.

target items. Fig. 3 shows the workflow of the triplet attention layer. In our model, the triplet attention layer comprises three modules that facilitate mutual interaction among the channel C , height H , and width W dimensions. When the image $\mathcal{T}$ is put to the triplet attention layer, it simultaneously undergoes both max pooling and average pooling operation, which demonstrates the interaction between the H and W dimensions. The outputs of the two pooling layers are then concatenated to create a matrix with a shape of $[2, H, W]$. This operation, called Z -Pool, aims to compress the image along the first dimension while retaining the important items in the other two dimensions, as shown in Eq. (2).

$$Z\text{-Pool}(\mathcal{T}) = [\text{MaxPool}(\mathcal{T}), \text{AvgPool}(\mathcal{T})]. \quad (2)$$

where, the channel dimension C is compressed into the Z -Pool, and the output shape is $[2, H, W]$. After that, the triplet attention layer performs convolution and sigmoid activation to generate weights that are used to readjust the input image $\mathcal{T}$.

The operations in the other two modules are similar, but the image $\mathcal{T}$ is rotated before compressing the H channel and W channel, respectively. Next, the three resulting outputs are combined and averaged, thereby expanding the model's receptive field and enhancing the cross-dimensional interactions of the original image. The process of obtaining the rebuilt image $\mathcal{T}'$ through the triplet attention layers is shown in Eq. (3).

$$\mathcal{T}' = \frac{1}{3} [\mathcal{T}_1 \nabla (\psi_1(\mathcal{T}_1^*)) + \hat{\mathcal{T}}_2 \nabla (\psi_2(\hat{\mathcal{T}}_2^*)) + \hat{\mathcal{T}}_3 \nabla (\psi_3(\hat{\mathcal{T}}_3^*))]. \quad (3)$$

where $\hat{\mathcal{T}}_2 \in \mathbb{R}^{H \times C \times W}$ and $\hat{\mathcal{T}}_3 \in \mathbb{R}^{W \times H \times C}$ represent the images obtained by flipping the first dimension with the second and third dimensions of the image $\mathcal{T}$, respectively; $\mathcal{T}_i^*$ represents the matrix after Z -Pool operation; the symbol ' ∇ ' represents the sigmoid activation function and the symbol ' ψ ' represents the two-dimensional convolutional operations.

2. Embedding layer

The embedding layer flattens the image $\mathcal{T}'$ into a two-dimensional matrix to meet the input requirements of the ViT model. The operations of patch segmentation, linear projection and embeddings are shown in Fig. 4. The patch segmentation operation divides the input image into fixed-size S patches. Let a_i represent a patch obtained after patch segmentation, which has a shape of $[C, S, S]$. The image after patch segmentation can be represented as a set of patches $\mathcal{T}' = [a_1, a_2, \dots, a_z]$, where $z = \frac{H \times W}{S^2}$. The linear projection operation has a hyper-parameter called the embedding dimension, denoted as D . Based on the size of embedding dimension n , each patch is linearly mapped into a vector a'_i with a shape of $[1, n]$. After that, all these vectors are concatenated to generate a two-dimensional matrix $A = [a'_1, a'_2, \dots, a'_z]$. This operation maps the input image to a high-dimensional space, enabling the

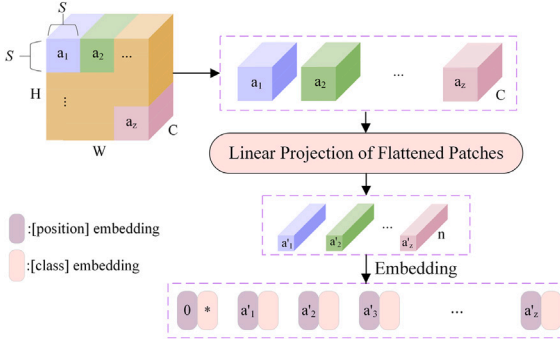


Fig. 4. Patch segmentation and linear projection.

model to learn more expressive feature representations. Next, the model concatenates a class embedding vector along the first dimension of this two-dimensional matrix. The shape of the matrix A is transformed from its original shape to a new shape of $[z + 1, n]$ by adding position embedding. The class embedding encodes the classification information of the encrypted traffic, while the position embedding encodes the positional information of each data point in the matrix. Both of them are trainable parameters. The embedding process is shown in Eq. (4).

$$A = [a'_1, a'_2, \dots, a'_z] \xrightarrow{\text{embedding}} [a^{cls}, a'_1, a'_2, \dots, a'_z] + E_{pos}, E_{pos} \in \mathbb{R}^{(z+1) \times n}. \quad (4)$$

where a^{cls} denotes the class embedding vector and E_{pos} is the position embedding vector.

3. Transformer encoder layer

Fig. 5 shows the structure of transformer encoder and MLP module. The two-dimensional matrix A undergoes a normalization operations to reduce internal covariate shift. Then, the multi-head self attention layer takes the input and uses weight matrices W^q, W^k, W^v to obtain the corresponding queries (q), keys (k), and values (v), as shown in Eq. (5).

$$q = AW^q, k = AW^k, v = AW^v. \quad (5)$$

Next, according to the number of attention heads (h), the acquired q , k , and v are divided evenly into h parts, resulting in three vectors $Q = [q^1, q^2, \dots, q^h]$, $K = [k^1, k^2, \dots, k^h]$ and $V = [v^1, v^2, \dots, v^h]$. After that, the multi-head self attention layer applies softmax activation function to assign the appropriate attention weights for each head, and then concatenates these attention weights. Then, combine them with the output weight matrix W^O to obtain the multi-head self-attention weight matrix. The process is shown in Eq. (6).

$$head_i = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i, \quad (6)$$

$$MSA(Q, K, V) = \text{Concat}(head_1, \dots, head_h) W^O.$$

where d_k is the dimension of the vector V , Q_i, K_i and V_i represent the i th element of the three vectors Q, K and V , W^O is a trainable parameter matrix.

The input embedded patches are connected to the multi-head self attention weight matrix through a residual connection. Residual connections can help mitigate the problems of gradient vanishing and exploding. The embedded patches are then passed into the MLP module. The MLP module consists of two linear layers, a GELU activation function, and a dropout layer. The number of nodes is quadrupled after the first linear layer, but then decreases again after the second linear layer. This is done to improve the model's ability to express more complex relationships between features. After the last operation in the MLP block, the model extracts the class embedding a^{cls} to generate

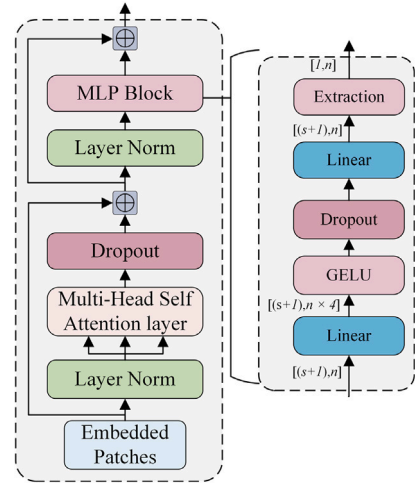


Fig. 5. The structure of transformer encoder.

a spatial feature vector $f_1 \in \mathbb{V}^{1 \times n}$. The whole process of transformer encoder layer is shown in Eq. (7).

$$A_L = \text{Drop}(MSA(LN(A_{L-1}))) + A_{L-1}, \quad f_1 = MLP(LN(A_L)) + A_L. \quad (7)$$

where L represents the number of stacked transformer encoder layers, Drop is the dropout operation and LN is the normalization layer.

3.2.2. Temporal feature extraction

In this section, the detailed descriptions about the temporal feature extraction process are provided, including gaussian noise addition, the embedding layer and the transformer encoder layer. It is worth noting that since the byte sequence B is used as the input for the temporal feature extraction of the model, b_i represents bytes rather than a vector, which differs from the input of the model's spatial feature extraction.

1. Gaussian noise addition

In the process of flattening the three-channel traffic image M into a one-dimensional byte sequence B , we added Gaussian noise to the sequence for several reasons. Firstly, noise is inevitable in network traffic data (due to transmission errors, packet loss, or signal interference). By introducing Gaussian noise, we can help the model better adapt to noisy environments in the real world, improving its generalization ability. Secondly, by adding an appropriate amount of Gaussian noise, we can "perturb" the data distribution to some extent, encouraging the model to focus more on the overall structure of the data rather than local details, thereby reducing feature loss caused by dimensional changes.

To calculate the Gaussian noise corresponding to each sequence, we obtained the probability density function $f(x)$ based on the mean μ and standard deviation σ of the sequence. Next, we randomly sample from the interval $[0, 1)$ to generate a uniform distribution random number sequence with the same length as the byte sequence. After that, the sequence is transformed into a Gaussian distribution sequence G using the Box-Muller transformation. The process is shown in Eq. (8).

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{(r-\mu)^2}{2\sigma^2}}, \quad (8)$$

$$B = B + G_i$$

where r is a random variable and it represents the processing time of the byte sequence. B represents the byte sequence B with Gaussian noise, G_i represents the Gaussian distribution sequence generated corresponding to the byte sequence.

2. Embedding layer

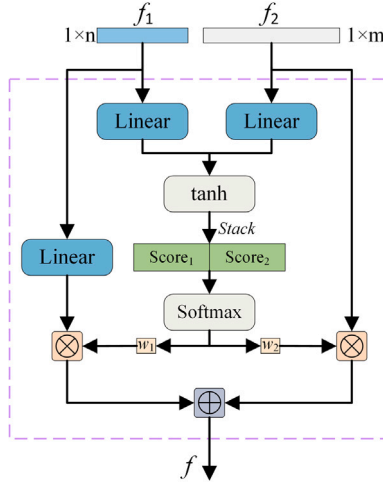


Fig. 6. The structure of feature fusion module.

The embedding layer consists of class embedding and position embedding to gather classification information and preserve the sequential nature of the data. Similar to the previously mentioned operations, the embedding layer adds class embeddings and position embeddings to the byte sequence $B = [b_1, b_2, \dots, b_m]$ to facilitate the extraction of classification information and maintain the positional relationships between the bytes. The whole process is shown in Eq. (9).

$$B' = [b_1, b_2, \dots, b_m] \xrightarrow{\text{embedding}} \begin{bmatrix} b_1 & b_2 & \dots & b_m \\ b_1^{cls} & b_2^{cls} & \dots & b_m^{cls} \end{bmatrix} + E'_{pos}, E'_{pos} \in \mathbb{R}^{2 \times m}. \quad (9)$$

where, b_i^{cls} represents the data at the i th element of the class embedding b^{cls} , and E'_{pos} denotes the position embedding for the byte sequence.

3. Transformer encoder layer

Multi-head self attention helps our model learn features of the byte sequences and handle various types of input data. Although the dimensions of byte sequences differ from those of three-channel images, the transformer encoder can still be used to extract temporal relationships within the byte sequences. The transformer encoder layer takes the byte sequences B' as input, and applies the multi-head self attention mechanism to process the input in parallel, producing a temporal feature vector $f_2 \in \mathbb{V}^{1 \times m}$. The process is shown in Eq. (10).

$$B'_L = \text{Dropout}(\text{MSA}(\text{LN}(B'_{L-1}))) + B'_{L-1}, \quad (10)$$

$$f_2 = \text{MLP}(\text{LN}(B'_L)) + B'_L.$$

where L represents the number of stacked transformer encoder layers, Drop is the dropout operation and LN is the normalization layer.

3.2.3. Spatial-temporal features fusion and classification

Spatial features f_1 and temporal features f_2 contribute differently to specific classification tasks. It is important to effectively combine these two feature vectors to enhance the model's classification performance. Here, we choose the additive attention mechanism-based feature fusion method. The reasons for choosing this feature fusion method are shown as below. Firstly, the additive attention mechanism enables weighted fusion. And it can preserve feature information and reduce dimensions. Besides, it can enhance expressive capability through nonlinear transformations and learnable parameters. Secondly, the attention weights can be visualized, which helps in understanding some features are more important for specific classification tasks.

The process of feature fusion is illustrated in Fig. 6. First, we defined a hidden dimension and mapped f_1 and f_2 to the same space dimension through linear transformations. Next, we calculated the attention scores

Table 2
Dataset description.

Dataset	#Flow	#Packet	#Label	Task	Size
USTC-TFC2016	9.85K	97.11K	20	T1	28×28
ISCX-VPN2016	3.12K	41.11K	7	T2	40×40
CIC-IOT2022	5.40K	320.71K	10	T3	$3 \times 16 \times 16$
CESNET-QUIC22	3.03M	–	18	T3	$3 \times 16 \times 16$

using the tanh activation function and a weight matrix. Finally, we performed a weighted fusion of the two feature vectors based on the attention scores. Through the above steps, we obtained a global feature f , which includes the spatial features and sequential temporal features extracted by our proposed model, EncryptoVision, from encrypted traffic. Finally, f is input into the classifier to compute the probabilities for each traffic category, denoted as Y . The process is shown in Eq. (11).

$$Y = \text{Softmax}(\omega \times f + \beta). \quad (11)$$

where ω represents the weight matrix and β represents the bias.

Besides, the loss function we used is the most common multi-classification loss function called the cross-entropy loss function $L(x, y)$, defined as Eq. (12).

$$L(x, y) = -\frac{1}{k} \sum_{i=1}^k \sum_{j=1}^c y_{ij} \log(Y(x_i)_j) \quad (12)$$

where k is the total number of samples, c is the total number of classes, y_{ij} is an indicator function that equals 1 if the true class of sample i is Y_j and 0 otherwise, and $Y(x_i)_j$ is the predicted probability that sample x_i belongs to class Y_j .

4. Experiments and performance evaluation

In this section, we introduce the datasets used in this paper, the evaluation metrics, the experimental environment, and the experiments conducted on our proposed model.

4.1. Datasets and experiment settings

4.1.1. Datasets description

To verify the classification performance of the model, we conducted experiments on four public encrypted traffic datasets: USTC-TFC2016, ISCX-VPN2016, CIC-IOT2022, and CESNET-QUIC22. The description for these four datasets is shown in Table 2. USTC-TFC2016 is a dataset aimed at studying the distinction between encrypted and unencrypted network traffic. The dataset includes various types of traffic, such as HTTPS, Email, FTP, and P2P, comprising 10 types of benign traffic and 10 types of malicious traffic. The ISCX-VPN2016 dataset features traffic characteristics from various applications (browsing, chat, email, streaming) over VPN, making it ideal for VPN traffic detection and classification. It includes 7 types of VPN traffic. CIC-IOT2022 includes traffic data from various IoT devices, as well as simulated encrypted network attack traffic (such as DDoS, malware, and data leaks). It includes 10 types of IOT traffic. CESNET-QUIC22 focuses on the emerging QUIC encryption protocol and contains traffic data from different application scenarios using the QUIC protocol. The dataset provides detailed features of the QUIC protocol, including connection establishment time, packet size distribution, and traffic encryption methods. It includes traffic from 102 cross-platform apps and 3 types of background traffic, which can be divided into 18 classes. Since the traffic data in the CESNET-QUIC22 dataset varies only by collection time, we have selected the data collected on November 27, 2022, for our experiment.

We conducted comparative experiments on these four datasets using different image sizes:

Table 3
Model evaluation metrics.

Metric	Formula
Accuracy	$Acc = \frac{TP+TN}{TP+TN+FP+FN}$
Precision	$P = \frac{TP}{TP+FP}$
Recall	$R = \frac{TP}{TP+FN}$
F_1 -score	$F_1 = \frac{2 \times P \times R}{P+R}$

- T1: Most existing computer vision-based methods convert traffic into single-channel images of size 28×28 . This size is small enough to reduce computational complexity while preserving sufficient feature information.
- T2: In Ref. [6], Zhao et al. proposed a new traffic transformation method, which transforms data from 5 packets into a single-channel image of size 40×40 . Specifically, each packet's header is represented by 2 header rows with 80 bytes, and 6 payload rows to accommodate the payload of each packet. This method expands the feature representation in both width and height, achieving significant performance improvements in their experiments.
- T3: We transform the traffic into three-channel RGB images with a size of $3 \times 16 \times 16$. The purpose is to expand the traffic features in the channel dimension, thereby enhancing the representation of the image features.

4.1.2. Performance metrics

To evaluate the classification model, We calculate the number of true positive (TP), true negative (TN), false positive (FP) and false negative (FN). Four evaluation metrics accuracy (Acc) precision (P), recall (R) and F_1 -score (F_1) are defined for every target class. The equations for these evaluation metrics are shown in Table 3.

Precision represents the proportion of samples predicted as positive by the model that are truly positive, reflecting the reliability of network traffic identification. Low precision indicates the inefficient use of network resources or unnecessary interference. Recall evaluates whether all true positive samples in the dataset are detected, reflecting the sensitivity of traffic classification. Low recall indicates threats to network security or service quality decrease. F_1 -score can be considered as the harmonic mean of model precision and recall, balancing both metrics, reflecting the comprehensive performance of the traffic identification and classification method, with higher values indicating better performance.

4.1.3. Experimental environment settings

The experiment is conducted on a Windows 11 64-bit operating system with 32 GB of memory. The system is equipped with an Intel (R) Core i5-13400 CPU @ 4.60 GHz and NVIDIA RTX 4070 GPU. Python 3.9.19 is used as the base environment language, and the deep learning framework is PyTorch. The EncryptoVision model is configured with the following parameters. The size of the traffic image is $3 \times 16 \times 16$, and the length of the byte sequence is 768. The convolutional layer used in the triplet attention layer has a kernel size of 7×7 , a stride of 1, and padding of 3. The hidden dimension size used for feature fusion is 512. The transformer encoder layer has a depth of 6, the multi-head self attention layer consists of 16 heads and the dropout rate is 0.1. Cross-validation is used with 10 fold. The source code is available at <https://github.com/Jsu-JYJ/EncryptoVision>.

4.2. Performance evaluation

For deep learning models, the setting of certain hyperparameters significantly affects the model's classification performance. In all experiments, we set the embedding dimension for the sequence feature extraction module to 768. Additionally, we used a learning rate of

Table 4
Test results with different hyperparameters.

Dataset	$S = 2$		$S = 4$		$S = 8$	
	Acc.	F_1	Acc.	F_1	Acc.	F_1
USTC-TFC2016	97.4%	0.974	98.1%	0.979	96.0%	0.959
ISCX-VPN2016	98.4%	0.961	99.4%	0.988	99.1%	0.979
CIC-IOT2022	99.0%	0.968	99.2%	0.974	97.9%	0.895
CESNET-QUIC22	91.3%	0.906	91.7%	0.911	91.0%	0.905
Dataset	$D = 64$		$D = 128$		$D = 256$	
	Acc.	F_1	Acc.	F_1	Acc.	F_1
USTC-TFC2016	87.4%	0.876	95.2%	0.951	98.1%	0.979
ISCX-VPN2016	94.0%	0.798	97.5%	0.933	99.4%	0.988
CIC-IOT2022	93.0%	0.719	98.6%	0.916	99.2%	0.974
CESNET-QUIC22	90.9%	0.903	90.5%	0.899	91.7%	0.911

0.001 and a batch size of 32. This subsection examines the impact of other hyperparameters in the model, such as patch size and embedding dimension for the spatial feature extraction module, on classification performance. The training process is designed to stop immediately as soon as the accuracy curve reaches convergence.

4.2.1. Patch size S

In the transformer architecture, the patch size is a very important hyperparameter. It affects the model's ability to perceive image details and global structure. A smaller patch size can capture more local details but also increases computational costs. Conversely, a larger patch size focuses more on global structure and large-scale features, but may lose some fine-grained information. Therefore, we conduct experiments with different patch sizes S of 2, 4, and 8, while keeping the embedding dimension D at 256. The experimental results are shown in Fig. 7.

From the changes in classification accuracy and loss values of the model under different patch sizes, it can be seen that the patch size does not have a very significant impact on the final classification performance. This is because, compared to images captured in the real world, the three-channel images constructed using the method in this paper are not only much smaller in size but also contain less rich features than those in the real world. Therefore, for these small-sized traffic images in this study, regardless of the chosen patch size, the model can map them into a similar two-dimensional space through linear mapping and dimensional embedding, encompassing most of the feature information in the images.

4.2.2. Embedding dimension D

The embedding dimension D is a core hyperparameter that defines the size of the two-dimensional space to which each input is mapped. A higher embedding dimension provides stronger representational capacity and better modeling of complex features and relationships, but it may also lead to longer training and inference times. To study its impact, we conduct experiments with embedding dimensions of 64, 128, and 256 across four datasets, while keeping other parameters fixed. As shown in Fig. 8. The embedding dimension has a significant impact on the model's classification performance. It can be observed that a smaller embedding dimension ($E = 64$) performs worse compared to larger embedding dimensions ($E = 128$, $E = 256$). Additionally, when the embedding dimension increases from 128 to 256, the improvement in classification accuracy is not as pronounced as that from 64 to 128. As mentioned earlier, the model maps the traffic images into a two-dimensional space, enriching the representation of the traffic. An embedding dimension that is too small ($E = 64$) is insufficient to capture the complex features and relationships within the model. Therefore, when $E = 128$, there is a significant improvement in classification accuracy. However, increasing the embedding dimension to 256 encounters the problem of diminishing returns, which means the model is approaching its saturation point in representational capacity. Further increasing the dimension will yield diminishing performance gains and

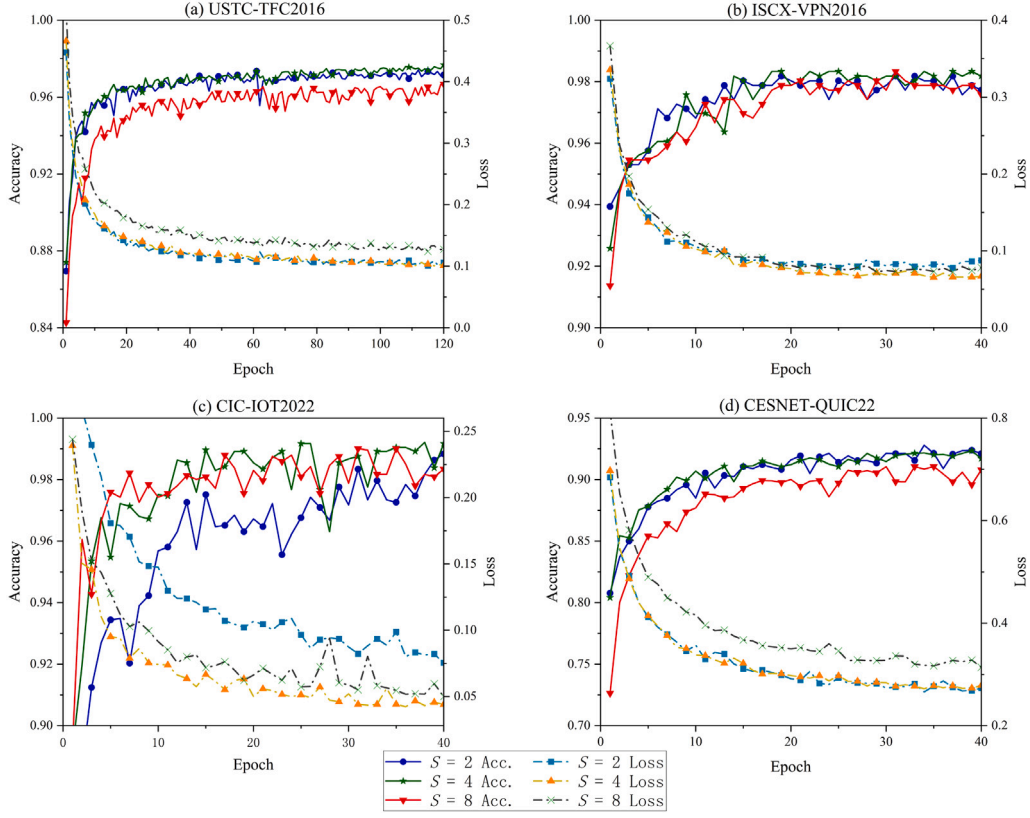


Fig. 7. Training performance of different patch size.

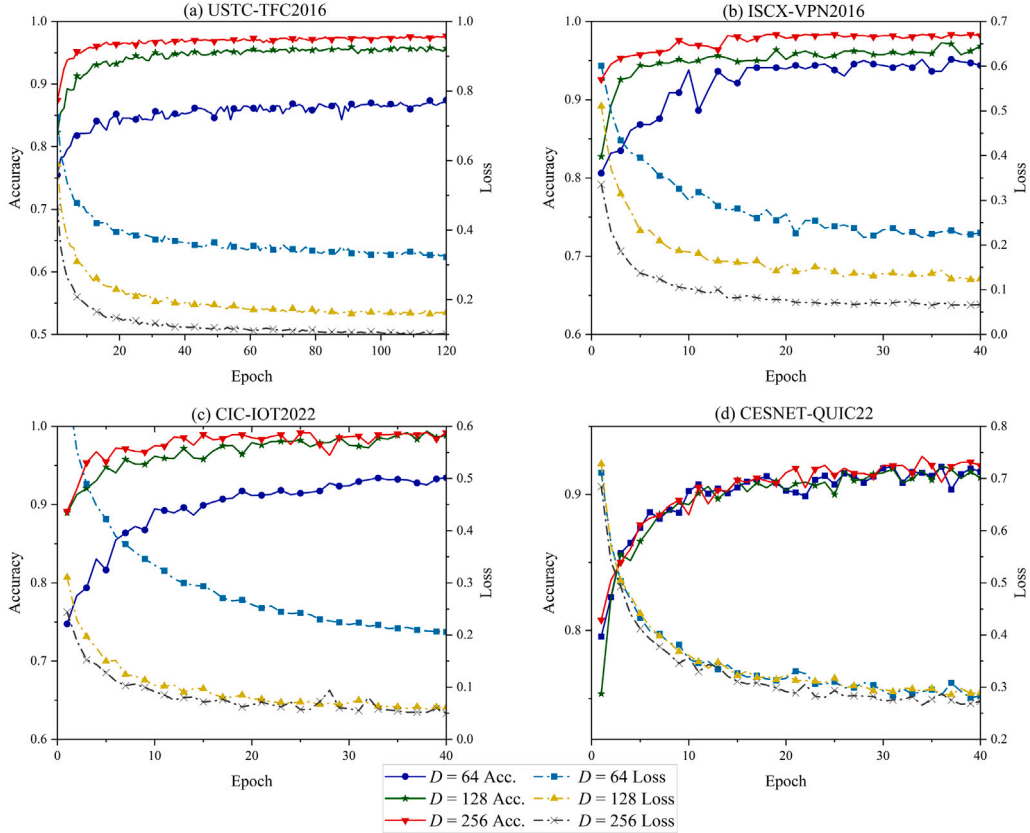


Fig. 8. Training performance of different embedding dimension.

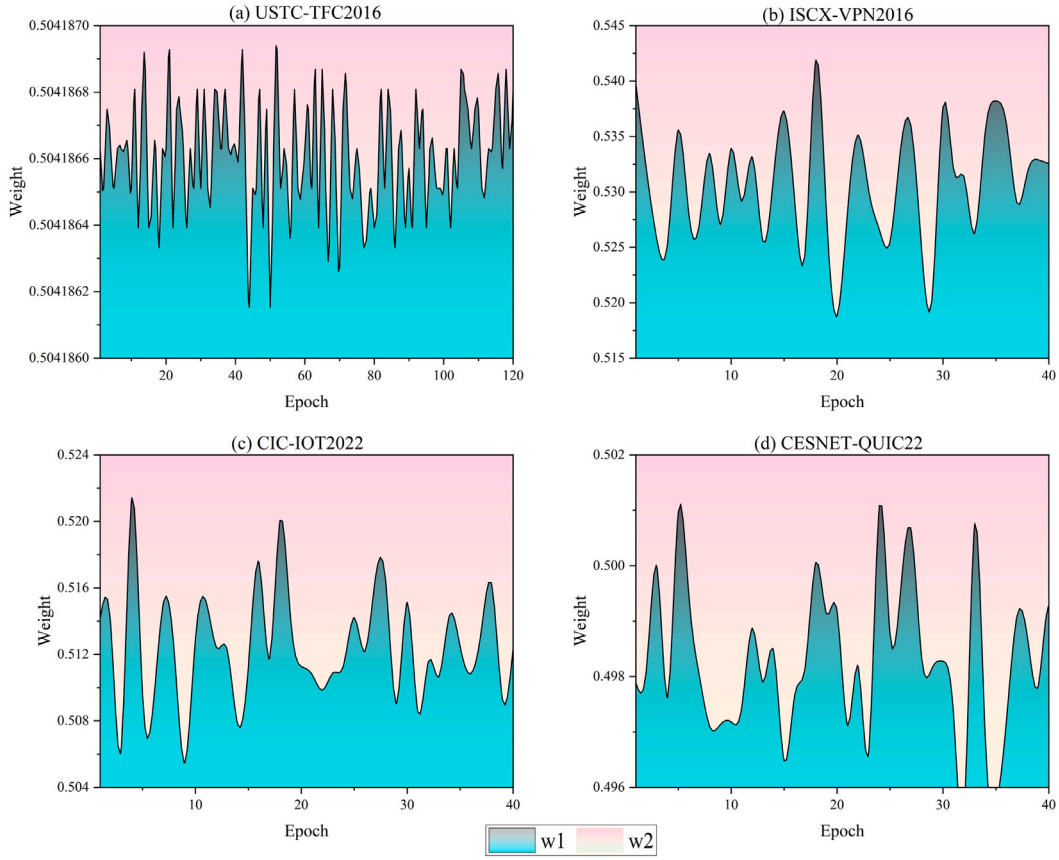


Fig. 9. Changes in dual-modal feature fusion weights on four datasets.

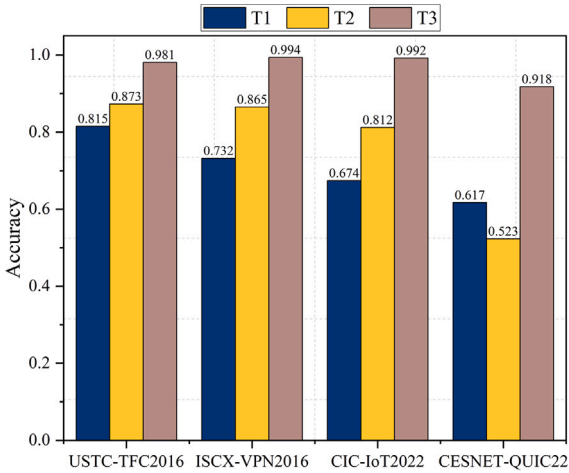


Fig. 10. Classification performance of different image size.

may even increase the risk of overfitting. Nevertheless, this is different for the CESNET-QUIC22 dataset, where the three chosen embedding dimensions have little impact on classification accuracy. This is because the dataset lacks long field values, resulting in traffic images that do not contain as rich features as those in other datasets. This means that when $E = 64$, the model is already capable of effectively extracting and analyzing the features included.

The experimental results on the test set for different patch sizes and embedding dimensions are shown in Table 4. It can be observed that when $S = 4$ and $E = 256$, the model performs the best overall. Therefore, we choose this set of parameters for subsequent experiments.

4.2.3. Image size

To validate the effectiveness of our proposed traffic representation method, we convert the four datasets used in the experiments into images of three different shapes: 28×28 (T1), 40×40 (T2), and $3 \times 16 \times 16$ (T3). The experimental results are shown in Fig. 10. From the results, it is evident that transforming the traffic data into three-channel RGB images yields significantly better classification performance compared to other image sizes. This improvement can be attributed to the richer representation provided by the three-channel format, which allows the model to capture more intricate patterns and relationships within the data. The enhanced performance also underscores the capability of our proposed model, EncryptoVision. This model is designed to extract features effectively from three-channel images, leveraging the triplet attention to enhance feature extraction processes. By employing a three-channel representation, we can encapsulate more information, leading to improved accuracy in classification tasks.

4.3. Dynamic weight changes in feature vector fusion

In our study, we also examine the feature fusion module within the model to demonstrate the significance of both the spatial feature extraction module and the temporal feature extraction module. The experimental results, shown in Fig. 9, provide insights into how these features contribute to the overall performance of the model across different datasets. For the USTC-TFC2016 dataset, we observe that the weights of the spatial features ($w1$) and temporal features ($w2$) remain relatively stable throughout the training process, with final values of approximately 0.50418 and 0.49582, respectively. The analysis of the ISCX-VPN2016 and CIC-IOT2022 datasets reveals more pronounced variations in feature weights. The importance of spatial features becomes significantly higher than that of temporal features,

Table 5
Ablation study of key components in our model.

Method	USTC-TFC2016		ISCX-VPN2016		CIC-IOT2022		CESNET-QUIC22	
	Acc.	F_1	Acc.	F_1	Acc.	F_1	Acc.	F_1
Max-Pool	97.6%	0.974	92.3%	0.795	97.9%	0.895	91.8%	0.912
Avg-Pool	97.8%	0.977	95.9%	0.957	97.2%	0.932	86.5%	0.868
w/o GN	90.1%	0.907	84.7%	0.805	88.1%	0.736	79.8%	0.770
w/o TA	85.2%	0.848	78.5%	0.470	87.9%	0.729	89.9%	0.886
w/o TFM	76.1%	0.763	97.5%	0.930	97.3%	0.884	75.4%	0.748
w/o SFM	96.7%	0.963	86.9%	0.408	92.8%	0.761	85.6%	0.871
Full model	98.1%	0.979	99.4%	0.988	99.2%	0.974	91.7%	0.911

with weights reaching 0.53255 and 0.51235 at the end of training, respectively. Moreover, for the CESNET-QUIC22 dataset, the observed weight changes are also minimal, with final weights of 0.499 for spatial features and 0.501 for temporal features.

The observed weight differences can be attributed to several potential factors. Firstly, each dataset has distinct characteristics and patterns. USTC-TFC2016 and CESNET-QUIC22 contains more uniform or predictable traffic data, resulting in more stable weights, while ISCX-VPN2016 and CIC-IOT2022 exhibit more complex behaviors and variations, leading to greater fluctuations in weights. Secondly, our proposed model places greater emphasis on extracting spatial features, which is another reason for the higher weights of spatial features in these datasets. Additionally, datasets like ISCX-VPN2016 and CIC-IOT2022 have class imbalances, causing the model to focus more on features that help distinguish the majority class, thus resulting in higher weights for spatial features in these cases. However, for the CESNET-QUIC22 dataset, the importance of temporal features exceeds that of spatial features. This is because the flow-level features in this dataset are not rich, and transforming them into traffic images results in an excessive amount of zero-filled data. As a result, the differentiation of traffic images is not very clear. In contrast, the temporal relationships within the sequence provide better support for classification.

4.4. Ablation study

In this section, we conduct ablation studies on the Z-pool module, Gaussian noise addition module (GN), the triplet attention module (TA), the temporal feature extraction module (TFM) and the spatial feature extraction module (SFM) within the model.

Firstly, we explore the necessity of the Z-pool module in data augmentation layer. As shown in Table 5, replacing the Z-pool module with max pooling and average pooling modules can lead to classification performance decline on the USTC-TFC2016, ISCX-VPN2016 and CIC-IOT2022 datasets. However, on the CESNET-QUIC22 dataset, using only the max pooling operation is slightly better. That is because the CESNET-QUIC22 dataset includes many statistical features of network flows. The experimental results demonstrate that in the dataset, emphasizing key traffic features through max pooling is more effective for classification than analyzing traffic features as a whole. Overall, using Z-pool including the max pooling and the average pooling can provide a more comprehensive representation of traffic features, and help improve traffic classification performance. Secondly, based on feature importance from previous model training across four datasets, we found that removing the TFM significantly reduced accuracy on the USTC-TFC2016 and CESNET-QUIC22 datasets by 22.0% and 16.3%, respectively. Eliminating the Gaussian noise addition module also caused accuracy drops of 18.0% and 11.9%. Conversely, for the ISCX-VPN2016 and CIC-IOT2022 datasets, where spatial features are more critical, removing the triplet attention mechanism resulted in even greater accuracy declines of 20.9% and 11.3%. The removal of SFM has a small impact on classification performance for the USTC-TFC2016 and CESNET-QUIC22 datasets, with accuracy decreasing by 1.4% and 6.1%, respectively. However, for the spatial feature sensitive datasets ISCX-VPN2016 and CIC-IOT2022, the classification accuracy decreased by

12.5% and 6.4%, respectively. Besides, there is a significant drop in the F_1 -score. The F_1 -score decrease by 0.580 and 0.213, respectively.

The ablation study highlights that each dataset exhibits distinct traffic patterns. The proposed traffic transformation and data augmentation methods (triplet attention and Gaussian noise addition) effectively emphasize the underlying features in the traffic, thereby enhancing classification accuracy. Furthermore, the ablation study demonstrates that both spatial and temporal features jointly influence the model's classification performance. Evaluating how the presence of one feature affects the other in traffic classification research is also crucial.

4.5. Performance comparisons

In this section, we compare the classification performance of our proposed model with a range of baselines and state-of-the-art models, namely 1D-CNN [20], ET-BERT [9], YaTC [6], ATVTSC [7], CNN-LSTM [36], FS-Net [8], AppScanner [37] and L2-BiTCN-CNN [5]. The introduction of these models and the reasons for selecting them as comparison models are shown as below.

AppScanner is a ML-based method using statistical features for traffic classification. 1D-CNN is the earliest end-to-end traffic classification model and is also the first to propose converting traffic into images for classification. CNN-LSTM adds an LSTM model on top of 1D-CNN to extract features from any three consecutive packets in the flow. FS-Net uses raw traffic information as sequential input to extract relevant features to improve classification performance. L2-BiTCN-CNN utilizes bidirectional TCN and CNN to extract temporal and spatial features from the traffic, and performs feature fusion to obtain global features for classification. ET-BERT uses the Datagram2Token method to convert session flows in network traffic into tokens, similar to those in NLP, for model pre-training and fine-tuning. YaTC introduces an innovative traffic transformation method (referred to as T3 in this paper) for classification. ATVTSC converts traffic into a single-channel image and uses bidirectional LSTM and transformer encoder to extract features from the traffic. These three models are based on the transformer architecture.

Table 6 shows the classification comparisons of the nine models on the USTC-TFC2016, ISCX-VPN2016, CIC-IOT2022 and CESNET-QUIC22 datasets. The classification performance of AppScanner on the USTC-TFC2016, ISCX-VPN2016, and CIC-IOT2022 datasets is not as good as that of other deep learning models. This is primarily due to its reliance on statistical features of flows, which limits its ability to analyze the payloads of encrypted traffic. The classification performance of 1D-CNN, CNN-LSTM, FS-NET and L2-BiTCN-CNN is lower than that of the transformer-based models (ET-BERT, YaTC, ATVTSC, and EncryptoVision). This is because encryption technology makes the models difficult to extract the feature relationships within the traffic data. The transformer encoder can capture long-range dependencies and automatically learn complex traffic patterns. Additionally, YaTC, ET-BERT, ATVTSC and EncryptoVision use more comprehensive representation methods for encrypted traffic. Therefore, ET-BERT, YaTC, ATVTSC, and our proposed model can perform more effective feature extraction and in-depth analysis of encrypted traffic data. It also illustrates the importance of the improvement of traffic representation method. We

Table 6
Comparisons results on USTC-TFC2016, ISCX-VPN2016, CIC-IOT2022 and CESNET-QUIC22 datasets.

Method	USTC-TFC2016		ISCX-VPN2016		CIC-IOT2022		CESNET-QUIC22	
	Acc.	F_1	Acc.	F_1	Acc.	F_1	Acc.	F_1
AppScanner	62.3%	0.595	78.6%	0.756	78.3%	0.785	87.4%	0.871
1D-CNN	91.7%	0.922	92.8%	0.815	90.6%	0.874	63.2%	0.618
CNN-LSTM	92.5%	0.924	92.9%	0.841	89.4%	0.867	65.6%	0.642
FS-Net	88.1%	0.864	87.6%	0.873	84.2%	0.839	81.7%	0.809
L2-BiTCN-CNN	92.9%	0.929	92.8%	0.820	91.3%	0.876	76.9%	0.745
YaTC	97.1%	0.977	97.4%	0.971	94.6%	0.913	–	–
ET-BERT	97.3%	0.973	98.8%	0.987	94.5%	0.945	87.7%	0.875
ATVITSC	95.5%	0.959	96.9%	0.938	97.1%	0.936	87.6%	0.861
Our model	98.1%	0.979	99.4%	0.988	99.2%	0.974	91.7%	0.911

observe our proposed model outperforms all the compared models on the four datasets, with its classification F_1 -score reaching up to 0.979, 0.988, 0.974 and 0.911 on the four datasets, respectively. This is because our well-designed feature representation method, dual-modal model structure and dynamic feature weight allocation method.

Among these transformer-based models, ET-BERT is a NLP-based model. ET-BERT converts encrypted traffic data into semantic sequences. This transformation method generates deep contextual representations, enabling the language model to extract fine-grained features and achieve F_1 -score of 0.973, 0.987, 0.945, and 0.875 on the four datasets. However, NLP-based models typically have more parameters than CV-based models. For example, our proposed EncryptoVision needs 48 million parameters, but ET-BERT needs 132 million parameters. The higher parameter count requires extra computational resources for training and inference. The experimental results show our model outperforms ET-BERT with fewer parameters. It can be concluded that transforming encrypted traffic into three-channel images is beneficial for enriching the feature representation of the traffic, and our proposed dual-modal fusion-based model can effectively classify encrypted traffic flows.

5. Conclusions and future work

This paper proposes a dual-modal fusion-based multi-classification model called EncryptoVision, which introduces channel attention mechanism into the ViT model for recognizing encrypted network traffic. The model transforms the encrypted traffic into three-channel images and uses triplet attention to enhance the interactions of the data in three channels, enabling the model to extract more fine-grained features. Next, multi-head self attention mechanism is applied to extract spatial and temporal features. Additionally, a spatial-temporal feature fusion technique is proposed to get the global feature for classification. The USTC-TFC2016, ISCX-VPN2016, CIC-IOT2022 and CESNET-QUIC22 datasets are used to conduct rigorous tests to evaluate the EncryptoVision model. The proposed model achieves F_1 -scores of 0.979, 0.988, 0.974 and 0.911 in the encrypted traffic multi-classification task on the respective datasets. This method outperforms state-of-the-art approaches in terms of classification performance.

Our work has two limitations and related future work. First, we tested our model using only four publicly available network traffic datasets. However, in real-world network environments, the volume and classifications of encrypted traffic are not predetermined. Thus, future studies should focus on enhancing the model's generalization ability and robustness across diverse datasets. Second, while we have demonstrated the use of a transformer encoder to extract temporal characteristics from the traffic data, this approach significantly increases the model's parameter count and prolongs the training process. In future work, we plan to develop a hybrid approach that combines a reservoir network with a simplified transformer encoder to extract temporal features while further reducing the parameter count.

CRedit authorship contribution statement

Zhiyuan Li: Writing – review & editing, Methodology, Funding acquisition, Conceptualization. **Yujie Jin:** Writing – original draft, Software, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by The National Key Research and Development Program project (2020YFB1005503); The Jiangsu Provincial Natural Science Foundation project (No. BK20201415).

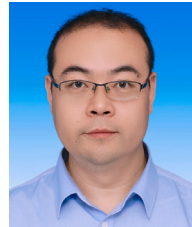
Data availability

Data will be made available on request.

References

- [1] R. Dingledine, N. Mathewson, P.F. Syverson, et al., Tor: The second-generation onion router, in: *USENIX Security Symposium*, vol. 4, 2004, pp. 303–320.
- [2] M.-H. Maras, Inside darknet: the takedown of silk road: Marie-helen maras reports on the unexplored underworld of cyberspace, *Crim. Justice Matters* 98 (1) (2014) 22–23.
- [3] A. Azab, M. Khasawneh, S. Alrabaaee, K.-K.R. Choo, M. Sarsour, Network traffic classification: Techniques, datasets, and challenges, *Digit. Commun. Netw.* 10 (3) (2024) 676–692.
- [4] R. Zhao, M. Zhan, X. Deng, F. Li, Y. Wang, Y. Wang, G. Gui, Z. Xue, A novel self-supervised framework based on masked autoencoder for traffic classification, *IEEE/ACM Trans. Netw.* 32 (3) (2024) 2012–2025.
- [5] Z. Li, X. Xu, L2-BiTCN-CNN: Spatio-temporal features fusion-based multi-classification model for various internet applications identification, *Comput. Netw.* 243 (2024) 110298.
- [6] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, Z. Xue, Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, 2023, pp. 5420–5427.
- [7] Y. Liu, X. Wang, B. Qu, F. Zhao, ATVITSC: A novel encrypted traffic classification method based on deep learning, *IEEE Trans. Inf. Forensics Secur.* (2024).
- [8] C. Liu, L. He, G. Xiong, Z. Cao, Z. Li, Fs-net: A flow sequence network for encrypted traffic classification, in: *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*, IEEE, 2019, pp. 1171–1179.
- [9] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, J. Yu, Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification, in: *Proceedings of the ACM Web Conference 2022*, 2022, pp. 633–642.
- [10] T.T. Nguyen, G. Armitage, A survey of techniques for internet traffic classification using machine learning, *IEEE Commun. Surv. Tutor.* 10 (4) (2008) 56–76.
- [11] D. Qin, J. Yang, J. Wang, B. Zhang, IP traffic classification based on machine learning, in: *2011 IEEE 13th International Conference on Communication Technology*, IEEE, 2011, pp. 882–886.
- [12] M. Soltani, M.J. Siavoshani, A.H. Jahangir, A content-based deep intrusion detection system, *Int. J. Inf. Secur.* 21 (3) (2022) 547–562.

- [13] O. Salman, I.H. Elhajj, A. Kayssi, A. Chehab, A review on machine learning-based approaches for internet traffic classification, *Ann. Telecommun.* 75 (11) (2020) 673–710.
- [14] J. Erman, A. Mahanti, M. Arlitt, I. Cohen, C. Williamson, Offline/realtime traffic classification using semi-supervised learning, *Perform. Eval.* 64 (9–12) (2007) 1194–1213.
- [15] Z. Mammeri, Reinforcement learning based routing in networks: Review and classification of approaches, *IEEE Access* 7 (2019) 55916–55950.
- [16] J. Zhang, C. Chen, Y. Xiang, W. Zhou, Y. Xiang, Internet traffic classification by aggregating correlated naive bayes predictions, *IEEE Trans. Inf. Forensics Secur.* 8 (1) (2012) 5–15.
- [17] R. Yuan, Z. Li, X. Guan, L. Xu, An SVM-based machine learning method for accurate internet traffic classification, *Inf. Syst. Front.* 12 (2010) 149–156.
- [18] D. Tong, Y.R. Qu, V.K. Prasanna, Accelerating decision tree based traffic classification on FPGA and multicore platforms, *IEEE Trans. Parallel Distrib. Syst.* 28 (11) (2017) 3046–3059.
- [19] A. Sharma, A.H. Lashkari, A survey on encrypted network traffic: A comprehensive survey of identification/classification techniques, challenges, and future directions, *Comput. Netw.* (2024) 110984.
- [20] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics, ISI, IEEE, 2017, pp. 43–48.
- [21] S. Soleymanpour, H. Sadr, M. Nazari Soleimandarabi, CSCNN: cost-sensitive convolutional neural network for encrypted traffic classification, *Neural Process. Lett.* 53 (5) (2021) 3497–3523.
- [22] X. Xiao, W. Xiao, R. Li, X. Luo, H. Zheng, S. Xia, EBSNN: Extended byte segment neural network for network traffic classification, *IEEE Trans. Dependable Secur. Comput.* 19 (5) (2021) 3521–3538.
- [23] W. Wei, H. Gu, W. Deng, Z. Xiao, X. Ren, ABL-TC: A lightweight design for network traffic classification empowered by deep learning, *Neurocomputing* 489 (2022) 333–344.
- [24] J. Zhang, Z. Xie, J. Sun, X. Zou, J. Wang, A cascaded R-CNN with multiscale attention and imbalanced samples for traffic sign detection, *IEEE Access* 8 (2020) 29742–29754.
- [25] F. Hu, S. Zhang, X. Lin, L. Wu, N. Liao, Y. Song, Network traffic classification model based on attention mechanism and spatiotemporal features, *EURASIP J. Inf. Secur.* 2023 (1) (2023) 6.
- [26] J. Hu, L. Shen, G. Sun, Squeeze-and-excitation networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 7132–7141.
- [27] S. Woo, J. Park, J.-Y. Lee, I.S. Kweon, Cbam: Convolutional block attention module, in: *Proceedings of the European Conference on Computer Vision, ECCV*, 2018, pp. 3–19.
- [28] D. Misra, T. Nalamada, A.U. Arasanipalai, Q. Hou, Rotate to attend: Convolutional triplet attention module, in: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2021, pp. 3139–3148.
- [29] M. Zhan, J. Yang, D. Jia, G. Fu, EAPT: An encrypted traffic classification model via adversarial pre-trained transformers, *Comput. Netw.* 257 (2025) 110973.
- [30] J. Cheng, Y. Wu, E. Yuepeng, J. You, T. Li, H. Li, J. Ge, MATEC: A lightweight neural network for online encrypted traffic classification, *Comput. Netw.* 199 (2021) 108472.
- [31] H.Y. He, Z.G. Yang, X.N. Chen, PERT: Payload encoding representation from transformer for encrypted traffic classification, in: 2020 ITU Kaleidoscope: Industry-Driven Digital Transformation, ITU K, IEEE, 2020, pp. 1–8.
- [32] B. Wang, Y. Su, M. Zhang, J. Nie, A deep hierarchical network for packet-level malicious traffic detection, *IEEE Access* 8 (2020) 201728–201740.
- [33] G.D. Gil, A.H. Lashkari, M. Mamun, A.A. Ghorbani, Characterization of encrypted and VPN traffic using time-related features, in: *Proceedings of the 2nd International Conference on Information Systems Security and Privacy, ICISSP 2016*, SciTePress, Setúbal, Portugal, 2016, pp. 407–414.
- [34] S. Dadkhah, H. Mahdikhani, P.K. Danso, A. Zohourian, K.A. Truong, A.A. Ghorbani, Towards the development of a realistic multidimensional IoT profiling dataset, in: 2022 19th Annual International Conference on Privacy, Security & Trust, PST, IEEE, 2022, pp. 1–11.
- [35] J. Luxemburk, K. Hynek, T. Čejka, A. Lukačovič, P. Šiška, CESNET-QUIC22: A large one-month QUIC network traffic dataset from backbone lines, *Data Brief* 46 (2023) 108888.
- [36] Z. Zou, J. Ge, H. Zheng, Y. Wu, C. Han, Z. Yao, Encrypted traffic classification with a convolutional long short-term memory neural network, in: 2018 IEEE 20th International Conference on High Performance Computing and Communications; IEEE 16th International Conference on Smart City; IEEE 4th International Conference on Data Science and Systems, HPCC/SmartCity/DSS, IEEE, 2018, pp. 329–334.
- [37] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic, in: 2016 IEEE European Symposium on Security and Privacy, EuroS&P, IEEE, 2016, pp. 439–454.



Zhiyuan Li received Ph.D. degree in Information and Communication Engineering at Nanjing University of Posts and Telecommunications, China. After PhD study, he continued postdoctoral research in Jiangsu High Technology Research Key Laboratory for Wireless Sensor Networks. From September 2016 to August 2018, he was an assistant research professor in the Department of Computer Science at Boise State University, USA. He is currently an associate professor in the School of Computer Science and Telecommunication Engineering at Jiangsu University, China. His research interests include computer networks and cyber security. He is a member of the IEEE and ACM and a senior member of China Computer Federation.



Yujie Jin is a graduate student at the School of Computer Science and Communication Engineering, Jiangsu University. His research interests include network traffic classification, encryption traffic identification and malicious traffic and feature extraction.